

分类号 TP391

学号 23023014

UDC 681

密级 公开

工学硕士学位论文

面向异构 GPU 集群的多模态大语言模型
并行训练策略研究

硕士生姓名 范锦山

学科专业 计算机技术

研究方向 分布式训练与优化

指导教师 李东升 研究员

协助指导教师 赖志权 副研究员

国防科技大学研究生院

二〇二六年三月

Parallel Training Strategy for Training Multimodal Large Language Models on Heterogeneous GPU Clusters

Candidate: Jinshan Fan

Supervisor: Prof. Dongsheng Li

Associate Supervisor: Prof. Zhiquan Lai

A dissertation

Submitted in partial fulfillment of the requirements

for the degree of Master of Engineering

in Computer Technology

Graduate School of National University of Defense Technology

Changsha, Hunan, P. R. China

March, 2026

独 创 性 声 明

本人声明所呈交的学位论文是我本人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表和撰写过的研究成果，也不包含为获得国防科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。

学位论文题目： 面向异构 GPU 集群的多模态大语言模型并行训练策略研究

学位论文作者签名： _____ 日期： _____ 年 _____ 月 _____ 日

学位论文版权使用授权书

本人完全了解国防科技大学有关保留、使用学位论文的规定。本人授权国防科技大学可以保留并向国家有关部门或机构送交论文的复印件和电子文档，允许论文被查阅和借阅；可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。

（保密学位论文在解密后适用本授权书。）

学位论文题目： 面向异构 GPU 集群的多模态大语言模型并行训练策略研究

学位论文作者签名： _____ 日期： _____ 年 _____ 月 _____ 日

作者指导教师签名： _____ 日期： _____ 年 _____ 月 _____ 日

目 录

摘 要	i
Abstract	iii
第一章 绪论	1
1.1 研究背景	1
1.1.1 训练多模态大语言模型面临的挑战	1
1.1.2 异构 GPU 集群的并行训练策略	3
1.2 研究现状	5
1.2.1 多模态大语言模型并行训练策略研究现状	5
1.2.2 异构 GPU 集群并行训练策略研究现状	6
1.3 研究内容与贡献	8
1.3.1 针对多模态大语言模型的复合粒度划分和异构设备映射	8
1.3.2 针对多模态大语言模型的提前终止流水线调度	8
1.3.3 针对多模态大语言模型的异构梯度检查点优化	8
1.4 论文组织结构	9
第二章 相关工作	11
2.1 多模态大语言模型	11
2.1.1 多模态大语言模型架构	11
2.1.2 多模态大语言模型训练数据	12
2.1.3 多模态大语言模型训练目标	12
2.2 并行与分布式训练基础	12
2.2.1 常见并行策略	12
2.2.2 多维混合并行	14
2.3 异构 GPU 集群并行训练策略	15
2.4 梯度检查点优化	17
第三章 基于锐度比策略的自适应锐度正则化算法 SAMAR	19
3.1 引言	19
3.2 SAMAR 算法设计与实现	19
3.2.1 锐度比策略设计	19
3.2.2 算法实现	19
3.3 SAMAR 算法的理论属性	19
3.3.1 基本假设	19

3.3.2 收敛性分析	19
3.4 实验验证	19
3.4.1 图像分类实验	19
3.4.2 Hessian Spectrum 分析	19
3.4.3 损失曲面可视化	19
3.5 本章小结	19
第四章 基于退火策略的自适应锐度正则化算法 ARSO	21
4.1 引言	21
4.2 ARSO 算法设计与实现	21
4.2.1 退火策略	21
4.2.2 算法实现	21
4.2.3 锐度比策略与退火策略的比较	21
4.3 ARSO 算法的理论属性	21
4.3.1 基本假设	21
4.3.2 收敛性分析	21
4.4 实验验证	21
4.4.1 GLUE 基准测试	21
4.4.2 神经机器翻译实验	21
4.4.3 损失曲面可视化	21
4.5 本章小结	21
第五章 总结与展望	23
5.1 全文工作总结	23
5.1.1 主要创新点	23
5.1.2 研究的局限性	23
5.2 未来工作展望	23
5.2.1 理论深化	23
5.2.2 应用拓展	23
致谢	25
参考文献	27
作者在学期间取得的学术成果	31
公开评阅信息	33
附录 A 模板提供的希腊字母命令列表	35

表 目 录

图 目 录

图 1.1	多模态大语言模型的核心组成部分	2
图 1.2	NVIDIA GPU 架构与代表性产品演进	4
图 1.3	使用 Megatron 训练多模态大语言模型（同构假设下的典型配置）	7
图 2.1	数据并行	13
图 2.2	张量并行	14
图 2.3	流水线并行	15
图 2.4	零冗余优化器 ZeRO	15

摘 要

国防科技大学是一所直属中央军委的综合性大学。1984 年, 学校经国务院、中央军委和教育部批准首批成立研究生院, 肩负着为全军培养高级科学和工程技术人才与指挥人才, 培训高级领导干部, 从事先进武器装备和国防关键技术研究的重要任务。国防科技大学是全国重点大学, 也是全国首批进入国家“211 工程”建设并获中央专项经费支持的全国重点院校之一。学校前身是 1953 年创建于哈尔滨的中国人民解放军军事工程学院, 简称“哈军工”。

关键词: 国防科技大学; 211; 哈军工

Abstract

National University of Defense Technology is a comprehensive national key university based in Changsha, Hunan Province, China. It is under the dual supervision of the Ministry of National Defense and the Ministry of Education, designated for Project 211 and Project 985, the two national plans for facilitating the development of Chinese higher education.

NUDT was originally founded in 1953 as the Military Academy of Engineering in Harbin of Heilongjiang Province. In 1970 the Academy of Engineering moved southwards to Changsha and was renamed Changsha Institute of Technology. The Institute changed its name to National University of Defense Technology in 1978.

Key Words: NUDT; MND; ME

符号使用说明

HPC	高性能计算 (High Performance Computing)
cluster	集群
Itanium	安腾
SMP	对称多处理
API	应用程序编程接口
PI	聚酰亚胺
MPI	聚酰亚胺模型化合物, N- 苯基邻苯酰亚胺
PBI	聚苯并咪唑
MPBI	聚苯并咪唑模型化合物, N- 苯基苯并咪唑
PY	聚吡咙
PMDA-BDA	均苯四酸二酐与联苯四胺合成的聚吡咙薄膜
ΔG	活化自由能 (Activation Free Energy)
χ	传输系数 (Transmission Coefficient)
E	能量
m	质量
c	光速
P	概率
T	时间
v	速度

第一章 绪论

1.1 研究背景

1.1.1 训练多模态大语言模型面临的挑战

近年来，人工智能技术发展迅速，应用场景不断拓展。深度神经网络（Deep Neural Network, DNN）在计算机视觉、自然语言处理、语音识别、工业检测、推荐系统等众多领域取得了突破性进展，已成为许多关键技术领域的核心基础 [1]。2017 年，Vaswani 等人提出基于自注意力机制（Self-Attention）的 Transformer 架构 [2]，摒弃了传统的循环与卷积结构，凭借并行化友好与长距离依赖的建模能力，迅速成为自然语言处理与跨模态建模的主流模型结构。

在自然语言处理（Natural Language Processing, NLP）方面，基于 Transformer 的预训练语言模型不断刷新各类基准。BERT[3] 通过双向编码与掩码预训练在理解类任务上取得显著优势；GPT-2[4]、T5[5] 等自回归或编码器 - 解码器模型则在生成与迁移学习上表现突出；GPT-3[6] 与后续的 GPT-4[7] 等大规模语言模型（Large Language Models, LLMs）更在少样本与零样本设定下展现出接近人类的泛化能力。在计算机视觉（Computer Vision）领域，Vision Transformer（ViT）[8] 及后续的规模化工作 [9] 表明：将图像切分为块（patch）并经 Transformer 编码，即可在 ImageNet 等基准上超越传统卷积网络。CLIP[10] 通过图文对比学习实现了开放词汇的视觉表征，DALL-E[11] 则展示了从文本到图像生成的可行性。上述进展共同表明，以 Transformer 为底座、以规模扩展为路径的技术路线已成为当前人工智能发展的主要范式之一。

多模态（Multimodality）指对两种或两种以上模态信息进行联合表示、推理与生成的技术范畴。在机器学习与人工智能中，常见模态包括文本、图像、视频、音频、表格以及各类传感器数据。多模态大语言模型（Multimodal Large Language Models, MLLMs）特指在参数规模巨大的语言模型基础上，融合对视觉、听觉等非文本模态的感知与生成能力的一类模型 [12–14]。这类模型既保留 LLMs 在语言理解、推理与生成上的优势，又能够直接处理图像、视频等多模态输入，实现“看图说话”“以文生图”等跨模态任务，与人类理想的人工智能模型相契合，被广泛视为迈向通用人工智能（AGI）的重要路径之一 [15]。

从能力上看，MLLMs 可完成图像描述、视觉问答、图文对话、文档理解、以及根据文本生成图像或视频等任务，在智能助手、自动驾驶、内容创作、教育培训、工业质检等场景具有广阔应用前景。从结构上看，MLLMs 通常包含以下核心

组件（见图1.1）：（1）*Modality Encoder*，负责将图像、视频等非文本输入编码为特征表示，常采用预训练的视觉模型（如 CLIP、ViT）作为 backbone；（2）*Input Projector*，将编码后的多模态特征映射到与 LLM 词嵌入对齐的语义空间，常见形式包括线性层、Q-Former[12] 等轻量模块；（3）*LLM Backbone*，即大规模语言模型主体，承担主要的理解与生成计算，参数量通常占整体模型的 80%–90%；（4）*Output Projector*（在需要多模态生成时），将 LLM 输出映射到与目标模态生成器对齐的表示空间；（5）*Modality Generator*（在需要多模态生成时），根据投影表示生成图像、音频等目标模态，常采用扩散模型（如 Stable Diffusion）等。可见，LLMs 处于 MLLM 架构的核心，多模态能力是在其基础上的扩展与增强。

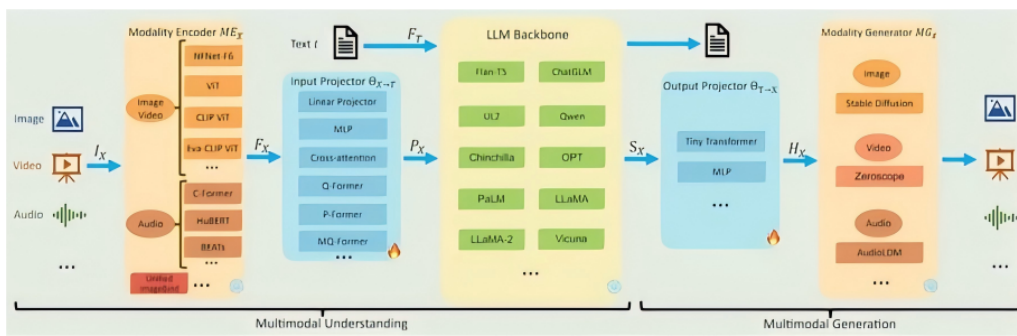


图 1.1 多模态大语言模型的核心组成部分

大量实证研究表明，模型性能随参数规模与训练数据规模的增加而持续提升，即存在所谓的“缩放律”（Scaling Laws）[16]。为追求更强性能，工业界与学术界不断推出参数量更大、数据规模更大的模型，从数亿参数（如 BERT）到千亿、万亿参数（如 GPT-3、DeepSeek-V3[17]、Qwen2.5[18] 等开源与闭源 LLMs），单卡或单机 GPU 的显存与算力已无法容纳整模型与完整训练流程 [19, 20]。相应的，多模态大模型的预训练与指令微调所需的数据量也急剧增长，从数百万图文对到数十亿级样本，单机算力、存储与 I/O 同样成为瓶颈。因此，在单计算设备上完成多模态大语言模型的训练已不现实，必须依赖分布式与并行训练，将模型参数、优化器状态、激活值与数据分布到多台设备上，并通过通信协同完成前向与反向计算，才能完成这样的大规模的模型的训练 [21, 22]。

目前，支撑大规模 MLLM 训练的主流并行策略可归纳为两大类。第一类是以计算与张量切分为核心的多维混合并行：数据并行 [23, 24] 在各设备上复制完整模型、按批次划分样本，通信集中在梯度 AllReduce，实现简单且在小模型上扩展性好；张量并行 [19, 20]（如 Megatron-LM）将单层内的矩阵按行或列切分到多卡，降低单卡显存并保留层内并行，是千亿级稠密模型的主流选择；流水线并行 [25, 26]（如 GPipe、PipeDream）将模型按层划分为若干阶段并分配到不同设备，

通过微批次与调度重叠计算与通信，缓解单设备层数过多的问题；序列并行 [19] 与专家并行 [27] 则分别在长序列注意力与混合专家（MoE）结构中沿序列维或专家维切分，与张量、流水线并行组合后可进一步扩展模型规模。第二类是以显存优化为核心的零冗余优化器系列：ZeRO[21, 22] 将优化器状态、梯度与参数分片存于各卡，前向与反向时按需聚合，在保持数据并行通信量的同时显著降低单卡显存；ZeRO-Infinity[22] 进一步将部分状态卸载至 CPU 与 NVMe，突破 GPU 显存墙；ZeRO++[28] 通过量化与通信重算减少跨节点通信量，提升多节点训练效率。上述并行策略常与混合精度训练 [29]、激活检查点 [30] 等技术结合，共同构成当前大模型与 MLLM 训练的基础设施。然而，上述策略在设计时多假设同构集群与单一模型形态，在面对异构 GPU 集群与模块异构的 MLLM 时，仍存在效率与可扩展性方面的挑战，这也是本文关注的重点之一。

1.1.2 异构 GPU 集群的并行训练策略

多模态大语言模型的训练效率与成本高度依赖底层硬件平台。目前，工业界与学术界普遍采用 NVIDIA GPU 作为大模型与 MLLM 训练的主要加速器，并通过多卡、多机集群进行分布式训练。然而，NVIDIA 约每 1–2 年迭代一代计算架构并推出多款不同定位的 GPU，导致显存容量、算力与互联方式持续分化，实际部署中的集群往往由多代、多型号 GPU 混合组成，形成显存异构、算力异构与通信异构并存的局面 [31, 32]。

图1.2 概括了近年来 NVIDIA GPU 的架构演进。2016–2017 年的 Pascal（如 P100）与 Volta（V100）奠定了数据中心 GPU 与 Tensor Core 的基础；2018 年 Turing（RTX 20 系列）引入消费级光追与 INT8 推理；2020 年 Ampere 架构推出 A100（40GB/80GB HBM2e）与消费级 RTX 30 系列（如 RTX 3090 24GB），支持 PCIe 4.0 与 NVLink 3.0，成为当前许多实验室与云平台的主力 [33]。2022 年 Hopper 架构的 H100（80GB/96GB HBM3）针对 Transformer 与 LLM 训练做了专门优化 [34]，并广泛配备 NVLink 4.0 与 NVSwitch，单机多卡带宽可达数百 GB/s；同年 Ada Lovelace 架构的 RTX 40 系列（如 RTX 4090 24GB）在消费级市场提供高性价比算力。2024 年起，Blackwell 架构的 B100/B200 等数据中心 GPU 逐步量产，采用 HBM3e、更高带宽的 NVLink5 与 PCIe 6.0，进一步拉大与旧代设备的性能与互联差距 [35]。按 NVIDIA 已披露的路线图，后续还将推出 Rubin（约 2026 年）、Rubin Ultra（约 2027 年），以及面向推理与能效深度优化的费曼（*Feynman*）架构（约 2028 年），后者将采用下一代 HBM 与 3D 堆叠内存等技术，进一步凸显代际间显存与互联的差异。与此同时，不同代际与型号在 PCIe 版本（3.0/4.0/5.0/6.0）、是否配备 NVLink/NVSwitch、以及 InfiniBand 与 RoCE 等节点间网络上的差异，使

得通信带宽与延迟在集群内分布极不均匀。

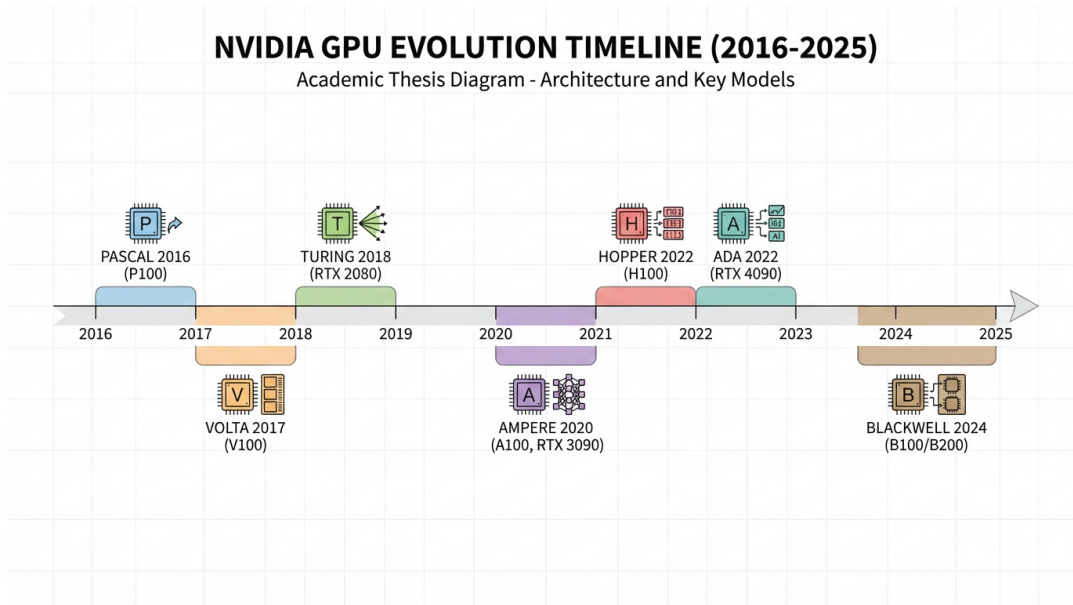


图 1.2 NVIDIA GPU 架构与代表性产品演进

在高校、研究所与中小企业的实际环境中，受预算与采购周期限制，集群常同时包含多代 GPU（例如部分 A100 或 H100 与大量 RTX 3090/4090、甚至更早的 V100/RTX 2080），单机内也可能混用不同显存与互联配置，难以做到“全集群同构”。而主流并行框架（如 Megatron-LM、DeepSpeed、PyTorch Distributed）在设计时多假设同构设备与统一拓扑 [19, 21]，在异构集群上直接使用往往导致：负载无法按算力与显存合理分配、通信成为瓶颈或部分高端卡空闲、以及因“木桶效应”整体效率下降 [31, 32]。因此，在显存、算力与通信三重异构的 GPU 集群上高效训练 *MLLM*，是许多机构共同面临且亟待解决的问题。

除了硬件设施固有的算力、显存和通信异构，模型异构也是现有并行训练框架无法高效训练多模态大语言模型的原因之一。负责将不同模态的输入数据编码为模型可理解的 Modality Encoder 一般是视觉模型 (如 CLIP、ViT 等)，核心模块 LLMs 是常见的基座大语言模型 (如 GPT-3、Llama-3[36])，具有庞大的参数量，大约占据整个多模态大语言模型 80%–90% 的参数量。Modality Generator 一般是一个 Diffusion 生成模型 (如 Stable Diffusion[37])，根据 LLMs 的输出产生图像、视频、音频等其它模态的数据。其中，LLMs 是由参数量很大的 Transformer 层堆叠而成，Encoder 可能由小的 Transformer 层或卷积层等其它神经网络组成，Generator 也可能是 U-Net[38] 等其他网络结构。前后两个 Projector 可能是 Transformer 层、Linear 层，也可能是 Q-Former[12] 等其它网络结构。多模态大语言模型的这种网络结构和传统的大模型结构不同，传统的大模型只有一种参数量固定的 Transformer 层，

每一层在特定 GPU 上运行所需的时间和显存都是固定的；而对于多模态大语言模型，模型的某一层变化多样，在不同的 GPU 上运行的时间开销和显存开销都不同，在异构 GPU 集群的环境下，这种模型异构对于异构集群上训练效率带来的影响被进一步放大。

由于深度学习的飞速发展，在计算机视觉、大语言模型、内容生成领域都涌现出很多性能强大的开源模型，为如今多模态大语言模型的训练范式奠定了基础。Blip-2[12] 率先提出利用已经训练好的 SOTA 模型，在训练中冻结部分模块，参数更新仅仅发生在非冻结的模块，模块之间使用一个小的神经网络连接起来进行模态转换，取得了非常好的效果。自此以后，基本上所有多模态大语言模型的训练都使用这种模式，比如近年来有代表性的 NextGPT[39]、DreamLLM[40] 等多模态大语言模型工作。除了冻结，多模态大语言模型的训练一般被分为多个阶段（模态对齐、指令微调等），不同的阶段需要冻结不同的模块，侧重赋予模型不同的功能。由于一个模块被冻结后，反向传播中的计算量会有所下降，根据实验观察这种下降程度会随着冻结位置的变化和模块自身神经网络的特性而有所不同，优化器也不再需要存储冻结的参数和可能的其余参数（比如使用 Adam[41] 优化器需要存储的梯度的动量和方差），这在很大程度上降低了模型训练所需要的显存。在异构 GPU 集群的环境下，冻结带来的算力、显存和通信影响进一步影响了多模态大语言模型的训练效率。

本课题将结合现有的并行策略和实际 GPU 集群的异构场景，基于 Megatron 等现有并行训练框架，针对多模态大语言模型中模型异构、冻结带来的影响以及实际 GPU 集群中存在的存储、算力和网络的异构情况，探索并实现一个高效的、细粒度的混合并行技术，提供一个面向异构复杂环境下多模态大语言模型并行训练的自适应编译优化方案，有效解决异构 GPU 集群下训练多模态大语言模型计算效率低、存储利用差、通信开销瓶颈等问题。

1.2 研究现状

1.2.1 多模态大语言模型并行训练策略研究现状

纵观近几年的人工智能领域的发展不难发现，目前的基座模型、视觉领域模型、内容生成模型正在不断向着大规模、大数据的方向纵深发展，这直接导致了多模态大语言模型也越来越大。由于 Scaling Law 的本身特性，经过实验证据表明，提升大模型的性能表现的两条主要途径是增大模型参数和扩大数据规模 [1]。通过提升模型参数，大模型可以有效地增强模型对于复杂场景的拟合能力和泛化能力；通过扩大数据规模，大模型可以获得更多的样本去学习充足的特征表示，从而提高其特征提取和表征能力 [23]，更好地适应于目前复杂的应用场景需要。

自 Transformer 提出以来，模型规模迅猛增长。从 2018 年 BERT 的 3.4 亿参数量，到 2019 年 GPT-2 的 15 亿参数量，再到 2020 年 GPT-3 的 1750 亿参数量，2021 年阿里巴巴 M6 的 10 万亿参数量等。在单个硬件加速器上训练大语言模型已经不再现实，并行训练是解决模型规模过大问题的必要途径。除此之外，由于大语言模型目前主要是数据驱动的，为了能够获得满意的性能表现，训练的数据量也在不断扩大。例如图片数据集 JFT-3B 包含接近 30 亿张图片，Common Crawl 网页数据中包含数千亿 tokens，存储图像数据集 Open Image[42] 需要 18TB 的存储空间，视频分析的 YouTube-8M[43] 数据集甚至需要高达 1PB 的存储空间。在单个硬件加速器上已经无法存储大模型的完整数据集，并行训练同样是解决数据集规模过大问题的必要方式。综上所述，模型参数量与数据集规模已经远远超出了单个设备的存储上限，并行训练是训练多模态大语言模型的必由之路。而由于 GPU 产品的快速更新换代、网络连接的不断发展变化，实际的并行训练平台中往往包含着许多不同存储、不同算力、不同带宽网络的异构设备，这些 GPU 及其网络之间的异构性会严重影响多模态大语言模型的训练性能。因此，面向异构 GPU 环境设计多模态大模型并行训练方法，实现自感知、自适应、细粒度的编译优化技术，进而提高算力资源的利用率、充分利用存储资源以及解决通信开销的瓶颈，对于未来的多模态大语言模型的训练是十分必要的。

1.2.2 异构 GPU 集群并行训练策略研究现状

正如前文所述，并行训练是当前工业界与学术界训练多模态大语言模型的主要手段，主流做法可概括为两类：以计算与张量切分为核心的多维混合并行，以及以显存分片为核心的零冗余优化器（ZeRO）系列。

多维混合并行通过在不同维度上划分模型或数据以降低单机负载，包括数据并行、张量并行、流水线并行、序列并行、上下文并行与专家并行等 [19, 20]。各方式可组合使用，在保证正确性的前提下通过通信协同完成前向与反向计算。零冗余优化器 ZeRO[21, 22] 在数据并行的基础上，将优化器状态、梯度与参数分片存于各卡，按需聚合，在通信量可控的前提下显著降低单卡显存，使千亿、万亿参数规模的训练成为可能。

NVIDIA 提出的 Megatron-LM[19, 20] 是多维混合并行的代表性框架，通过联合使用张量并行、流水线并行与数据并行，已支撑在数千张 GPU 上训练千亿至万亿级参数的大语言模型。Megatron-LM 的后续工作持续针对大模型训练的显存与效率进行优化：Korthikanti 等在 MLSys 2023 上发表的论文 [44] 针对超大 Transformer 的激活显存与重计算开销，提出序列并行技术，与张量并行协同使用，弥补了单一张量并行无法在归一化层和 Dropout 层切分中间激活值的不足。

然而，Megatron-LM 在设计时以同构 GPU 集群与单一形态的 Transformer 模型为前提，未考虑设备异构与模型异构。在实际异构环境下使用 Megatron 训练多模态大语言模型时，往往只能将不同型号、不同显存与带宽的 GPU 当作同构设备统一配置，其典型并行策略如图1.3所示。对于 MLLM 中结构各异的 Modality Encoder、LLM Backbone、Projector 与 Modality Generator，Megatron 也缺乏针对性的细粒度划分与映射：编码器与生成器的并行方式通常被动跟随 LLM 主干的划分，难以根据各子模块的算力与显存需求、以及不同 GPU 的 capability 做自适应分配。其结果是，显存小、算力弱或通信慢的 GPU 成为瓶颈，高端 GPU 无法充分发挥，整体集群利用率与训练效率下降 [31, 32]。

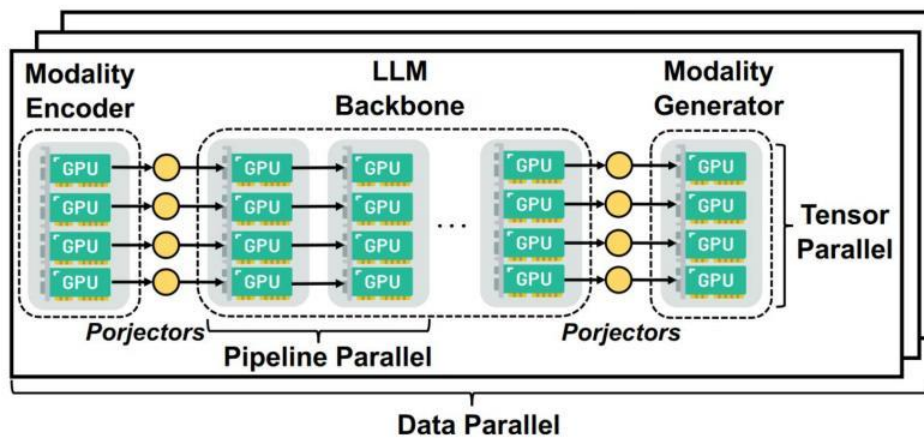


图 1.3 使用 Megatron 训练多模态大语言模型（同构假设下的典型配置）

ZeRO[21, 22] 由微软 DeepSpeed 团队提出，通过优化器状态、梯度与参数的分片与按需聚合，在数据并行框架下降低显存并提升可扩展性。但在面向多模态大语言模型的异构集群场景中，ZeRO 仍存在明显局限。一方面，ZeRO 的划分与通信模式与具体硬件的算力、显存及拓扑解耦，编译与调度阶段无法感知算力异构与显存异构，因而难以针对不同设备做负载与显存的均衡分配，易造成算力与存储的浪费。另一方面，在大规模分布式训练中，通信量与设备数量同步增长，而 ZeRO 未针对异构网络（如混合 NVLink、PCIe 与 InfiniBand）做拓扑感知的通信调度，在异构网络环境下容易形成通信瓶颈，限制整体训练吞吐。因此，在异构 GPU 集群上高效训练 MLLM，仍需在 ZeRO 与 Megatron 等现有框架基础上，引入对设备能力与模型结构的感知与自适应优化。

1.3 研究内容与贡献

1.3.1 针对多模态大语言模型的复合粒度划分和异构设备映射

本课题结合现有并行策略的不足,考虑实际 GPU 集群的异构场景,针对实际 GPU 集群中存在的存储、算力和网络的异构情况,旨在设计并实现一个高效的、细粒度的混合自动并行编译优化技术,提供一个面向异构复杂环境下多模态大语言模型并行训练的自适应编译优化方案,有效解决异构 GPU 集群下计算效率低、存储和通信开销瓶颈等问题,提升多模态大语言模型并行训练的效率。

针对异构 GPU 设备环境下的存储异构导致的存储空间利用不均衡的问题、通信异构带来的通信瓶颈问题、算力异构带来的 GPU 利用率不高的问题、模型异构带来的算力分配问题、冻结对于显存和计算需求的变化问题,本课题提出一种综合的、面向异构设备和异构模型的多模态大语言模型计算图的层级划分方法,通过为不同属性的计算设备分配不同大小的部分模型,实现计算资源、存储空间、通信带宽的充分利用,争取最大限度提高训练速度。

1.3.2 针对多模态大语言模型的提前终止流水线调度

MLLM 训练与常见的大语言模型的训练过程不同,通常需要分为不同的阶段进行训练,在不同的阶段中对不同模块进行冻结,冻结部分的反向计算量远小于未冻结部分,进一步加剧阶段间执行时间的不均衡,使得气泡率在异构流水线中居高不下,而且可能存在冗余的计算和通信。

本课题针对上述问题,提出一种面向多模态大语言模型的提前终止流水线调度策略。该策略可以感知模型在不同阶段的冻结情况,根据各阶段的冻结情况,调整模型的划分策略;同时结合冻结特性,检测冗余的计算和通信操作并且直接丢弃,提高多卡流水线的有效算力利用率与训练吞吐。

1.3.3 针对多模态大语言模型的异构梯度检查点优化

激活检查点(梯度检查点)通过在前向时丢弃部分中间激活、在反向时按需重算,以计算换显存,是支撑大模型与 MLLM 在有限显存下训练的重要手段。然而,现有梯度检查点策略多采用全局统一的配置(如对全部层或按固定间隔插入检查点),未考虑异构集群中设备间显存与算力的差异:显存紧张、算力相对充裕的设备适合更激进的检查点以释放显存;显存宽裕、算力紧张或通信繁忙的设备则不宜过度重算,否则会加重计算或通信瓶颈。此外,MLLM 各子模块(Encoder、LLM、Projector、Generator)的层数、每层激活体积与计算量各不相同,在同一设备上对不同模块采用相同检查点策略也难以达到最优。

本课题针对上述不足，提出一种面向异构设备与异构模型的梯度检查点优化方法。该方法在给定层级划分与设备映射的基础上，以各设备的显存上限、算力及通信负载为约束，为不同设备上的不同模型阶段分别搜索或配置差异化的检查点策略（如检查点插入的粒度与密度），在满足显存不溢出的前提下，最小化因重计算带来的额外时间开销，并兼顾流水线各阶段的负载均衡。通过将梯度检查点与前述的复合粒度划分、流水线调度相结合，实现计算、存储与通信在异构环境下的联合优化，进一步提升 MLLM 在异构 GPU 集群上的训练效率。

1.4 论文组织结构

本论文共分为五章，各章内容安排如下：

第一章为绪论，介绍了面向异构 GPU 集群的多模态大语言模型并行训练的研究背景与意义，分析了多模态大语言模型面临的挑战与异构 GPU 集群并行训练策略的现状，并概括了本文的主要研究内容与贡献。

第二章为相关工作，从多模态大语言模型架构与训练、并行与分布式训练基础、异构 GPU 集群并行训练策略以及梯度检查点优化等方面对国内外相关研究进行综述。

第三章、第四章分别介绍本文提出的面向异构设备与模型的计算图划分方法、流水线调度方法、梯度检查点优化方法及实验验证。

第五章为总结与展望，对全文工作进行总结，并指出未来可能的研究方向。

第二章 相关工作

本章从多模态大语言模型、并行与分布式训练基础、异构 GPU 集群并行训练策略以及梯度检查点优化等方面对国内外相关研究进行综述。如今，基于 Transformer 的大语言模型在包括自然语言处理 [6, 7]、图像识别 [8]、多模态 [10]、图学习与推荐系统等众多前沿领域和下游任务上大放异彩。然而，随着模型参数的增加和数据规模的扩大，模型结构越来越复杂、模型参数量越来越大、训练数据越来越多，导致单个设备已经无法完成模型和数据的存储，多模态大语言模型的训练越来越有难度。为了解决上述问题，目前学界已经提出并应用了多种并行策略来实现和优化大模型的训练。然而，目前传统的并行策略往往忽略了并行训练平台的异构性，由于服务器集群中常常存在多种型号的 GPU 和网络连接，导致集群中存在着存储异构、算力异构和网络异构。在这样的复杂集群环境下，传统的并行策略会因为缺乏对异构环境的感知和适应而导致性能表现急剧下滑。因此，下文将主要从现有大模型的并行训练策略出发，对并行训练的研究现状进行详细论述，并讨论各个并行策略在异构环境下的优缺点。

2.1 多模态大语言模型

2.1.1 多模态大语言模型架构

多模态大语言模型 (MLLMs) 通常由 Modality Encoder、Input Projector、LLM Backbone、Output Projector 等模块组成。Modality Encoder 负责将不同模态的输入数据（如图像、视频、音频）编码为模型可理解的表示，常采用预训练的视觉模型如 CLIP[10]、ViT[8] 等。LLM Backbone 是模型的核心，通常为参数量巨大的 Transformer 基座模型（如 GPT 系列 [4, 6]、LLaMA[36]、T5[5] 等），占据整个模型约 80%–90% 的参数量。Modality Generator 一般为 Diffusion 类生成模型（如 Stable Diffusion[37]），根据 LLM 的输出生成图像、视频等其它模态数据。前后两个 Projector 用于将不同模态映射到共享语义空间，可能是 Linear 层、Transformer 层或 Q-Former[12] 等结构。Blip-2[12] 率先提出冻结预训练 Encoder 与 LLM、仅训练轻量 Projector 的范式，后续 NextGPT[39]、DreamLLM[40] 等工作均沿用并发展了此类架构。多模态大语言模型的这种模块异构性使得不同层在算力、显存和通信上的需求差异很大，在异构 GPU 集群上部署时面临独特的挑战。

2.1.2 多模态大语言模型训练数据

为获得满意的性能，大语言模型与多模态大语言模型通常依赖大规模数据训练。例如图像数据集 JFT-3B 包含约 30 亿张图片，Common Crawl 网页数据包含数千亿 tokens，Open Image[42] 需约 18TB 存储，YouTube-8M[43] 等视频数据集甚至需要 PB 级存储。在单个硬件加速器上既无法存储完整模型参数，也无法容纳如此大规模的训练数据，因此并行训练（数据并行、模型并行及其组合）成为必然选择。数据规模与模型规模的持续增长，也使得对并行策略的通信效率、存储划分和异构适应性提出了更高要求。

2.1.3 多模态大语言模型训练目标

多模态大语言模型的训练通常分为多个阶段，如模态对齐、指令微调等，不同阶段会冻结不同模块（如冻结 LLM 仅训练 Projector，或冻结 Encoder 与 LLM 仅训练部分层）。冻结会显著改变各模块的反向计算量与显存占用（如优化器状态仅需为未冻结参数维护），并影响流水线中各 stage 的负载均衡。在异构 GPU 集群下，这种因阶段和冻结策略带来的“模型异构”与设备异构叠加，进一步增加了并行策略设计与自动调优的难度。

2.2 并行与分布式训练基础

2.2.1 常见并行策略

数据并行（Data Parallelism）被广泛用于处理数据规模过大的情况，其核心思想是通过将训练数据在所有计算设备上划分，而每个计算设备都持有一份完整的模型参数 [23, 24, 45]。如图2.1所示，在训练过程的每次迭代中，每个设备可以独立地使用本地模型参数和批次数据计算对应批次的梯度；然后，所有设备在下次迭代之前进行通信操作，将各自计算得到的梯度数据进行同步，从而保证模型参数的一致性和结果的正确性。数据并行训练因其隐式性和直观性而成为并行训练最早提出的策略，也是实际应用中最常见的方式。PyTorch 框架中提供了 DistributedDataParallel[23] 工具用于数据并行训练。Horovod[24] 将数据并行策略适配到了多个框架中。Parallax[46] 则根据模型的特点灵活地使用参数服务器（Parameter Server）或 All-Reduce 通信原语进行数据同步，优化了数据并行中的通信开销。然而，数据并行存在着明显限制：每个 GPU 都保存模型权重的完整副本，因此不能直接用于训练具有大量参数的模型，限制了模型规模的扩展；同时，数据并行也将模型训练的工作限制在了单个计算设备内，无法通过设备并行的方式加速训练过程。

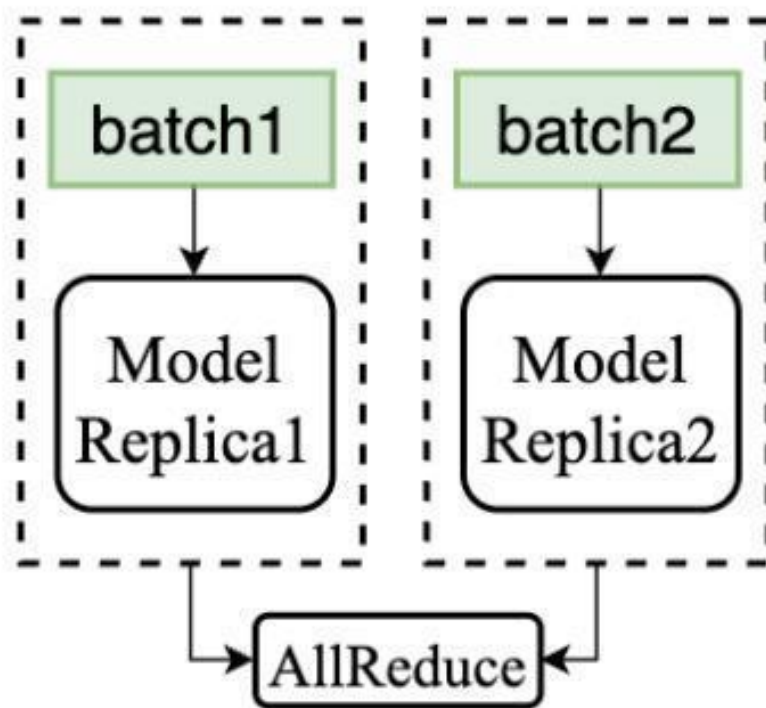


图 2.1 数据并行

模型并行 (Model Parallelism) 是实现大模型训练的有效途径, 包括张量并行 (Tensor Parallelism) [20] 和流水线并行 (Pipeline Parallelism) [25]。张量并行的核心思想是将模型在层内进行划分, 每个计算设备分别持有模型一部分的层内张量, 它们组合在一起是一份完整的张量, 从而将模型的参数和计算任务分别绑定到不同的相互连接的设备上。如图2.2所示, 在训练过程的每次迭代中, 每个计算设备只负责模型一部分参数的计算, 然后通过中间激活值和梯度值的通信, 使模型可以进行下一层的计算。Megatron-LM[20] 提出了针对 Transformer 模型张量并行的训练方法, 对每两个连续层的权重首先按行 (输入维度) 划分, 再按列 (输出维度) 划分, 消除了同步第一层中间输出的需要, 但随后仍然需要大量的跨设备通信。GShard[27] 提出了在计算图层面进行张量并行优化的思路。然而, 目前的张量并行方式受限于 GPU 服务器集群中的带宽情况, 尤其是跨节点带宽, 其并行规模无法进行大规模扩展, 往往需要结合其他并行方式一起加速。

流水线并行的核心思想是将模型在层间进行划分, 每个计算设备分别持有整个模型的若干层, 称为一个 stage; 这些 stage 组合起来是一份完整的模型。如果直接将不同设备上的 stage 串行地进行训练, 则会导致每个 stage 训练过程中只有一个设备在进行计算, 设备利用率极低。如图2.3所示, 流水线并行针对训练数据进一步划分为多个 micro-batch, 依次输入到模型中进行计算。尽管各级流水线在

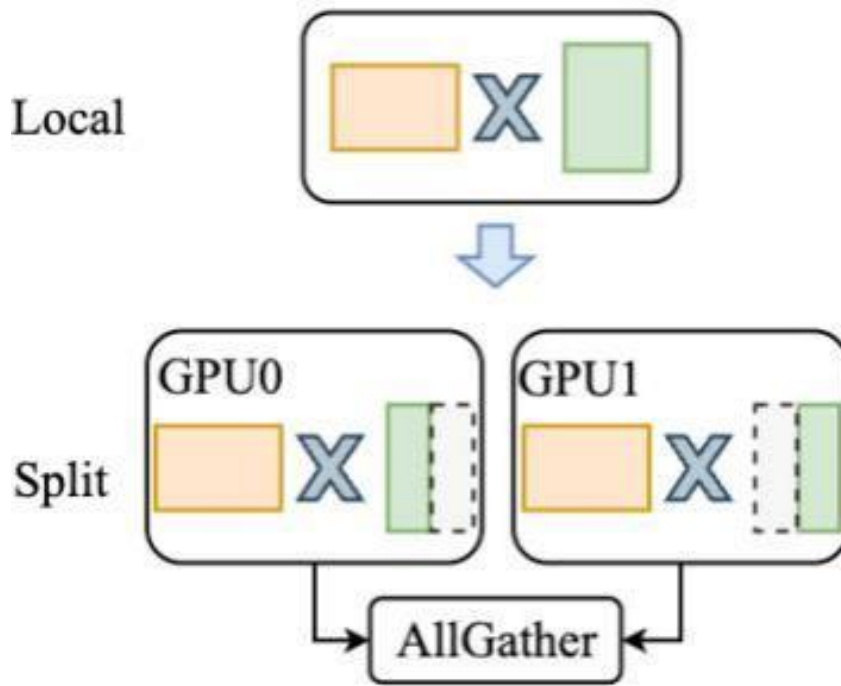


图 2.2 张量并行

不同的 micro-batch 之间存在数据依赖，然而 batch 之间数据是相互独立的，从而每个 micro-batch 可以依次进行模型训练，实现流水线的训练过程，大大减少计算设备的空闲时间。GPipe[25] 提出了使用 micro-batch 的流水线并行方式，通过设计多级流水减少了 Bubble 的空闲。PipeDream[26] 提出了 1F1B（1 次前向 1 次反向），通过异步调度的方式进一步减少了 Bubble 的空闲。TeraPipe[47] 则在自回归模型的单个训练 sequence 中使用 token 级流水线并行，实现了更细粒度的流水线并行。数据并行、张量并行和流水线并行策略相互正交，可以一起用于并行训练。由于流水线并行的通信量一般较小，且跨节点通信明显慢于节点内通信，因此张量并行往往仅限于节点内训练，而流水线并行则用于跨节点训练，以更好地利用硬件带宽 [20]。

2.2.2 多维混合并行

ZeRO[21] 是微软 DeepSpeed 团队提出的零冗余优化器技术，针对大规模深度神经网络中数据并行训练进行高效优化。如图 2.4 所示，在模型训练中，各部分的存储占用从高到低依次为优化器状态、梯度和参数。ZeRO 通过将以上三部分分别划分到每个计算设备中，进而减少存储开销和提高 GPU 利用率来扩展大语言模型的训练。在训练过程中，ZeRO 通过前向和反向的多次通信同步数据，保证训练结果的一致性。目前，学界已经提出了一些针对 ZeRO 的通信方面的改进工作。MiCS[48] 是在云服务环境下针对 ZeRO 的通信数据量进行减少和优化；

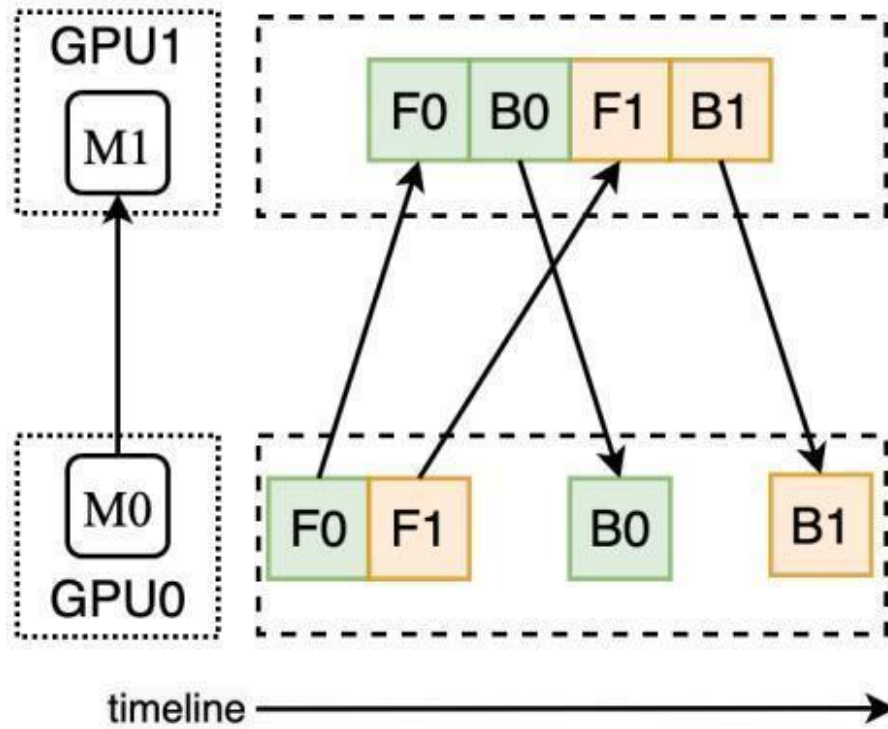


图 2.3 流水线并行

ZeRO++[28] 基于量化技术减少 ZeRO 的通信数据量；AMSP[49] 则将 ZeRO 的划分方式扩展到一个统一划分空间，寻找通信开销最小的策略，进一步扩展了大模型训练的规模。

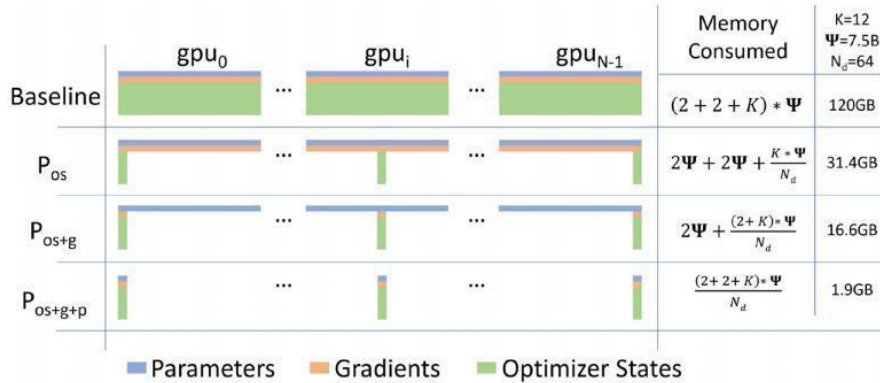


图 2.4 零冗余优化器 ZeRO

2.3 异构 GPU 集群并行训练策略

在存储方面，由于不同 GPU 之间的存储不一致，传统的并行策略会导致存储负载不均衡，大存储的设备会出现存储利用不充分的问题，影响了模型训练的扩展性。在算力方面，由于不同 GPU 之间的算力不一致，传统并行策略会导致“木

桶效应”，不仅使得计算资源利用率下降，还导致整体训练速度的降低。在网络方面，由于集群中存在的不对称的多级网络连接，使得不同设备之间的通信开销相去甚远，低带宽网络对应的通信将成为训练的性能瓶颈。因此，面向异构 GPU 集群的并行训练策略成为近年来的研究热点。

HetPipe[32] 结合了流水线并行（PMP）和数据并行（DP），多个 GPU 被分组为虚拟工作节点（VW），以流水线方式处理小批量数据，然后这些虚拟工作节点再通过数据并行提高训练的可扩展性和性能。但是 HetPipe 没有考虑虚拟工作节点内部的异构设备排列方式，对于通信开销和显存开销的考虑过于直观，而且完全没有考虑模型异构，在如今的多模态大语言模型训练时不适用。

Whale[31] 是阿里巴巴团队提出的一种面向异构存储的 GPU 集群的并行训练优化方法。由于实际 GPU 集群中存在的存储异构和算力异构问题，Whale 提出了一种基于硬件感知的负载均衡算法，通过按比例分配的方式为每个设备分配不同大小的批次数据和模型部分，实现了数据并行与张量并行的负载均衡。然而，Whale 没有考虑集群中普遍存在的网络异构性，对于存储异构和算力异构的优化方法过于直观和简单，并且是在子图的粗粒度下进行的划分和编译，也没有考虑 ZeRO 的并行方法，因此仍然需要进一步的优化。

AMP[50] 是卡耐基梅隆大学团队提出的一种针对异构环境下大模型的并行训练优化方法。AMP 同时考虑了模型异构和设备异构，针对模型中不同层的结构不同的情况进行了抽象和建模，而针对集群中不同 GPU 存储异构和网络异构进行了近似计算。在此基础上，AMP 基于 3D 并行方法（DP+TP+PP），设计了一个详细的开销模型（Cost Model）以评估异构模型在异构环境下的计算和通信开销，并通过动态规划算法确定最优的并行策略。尽管 AMP 在传统的 3D 并行策略的基础上提升了异构环境下的性能表现，但是 AMP 的并行维度是基于全局的粗粒度划分，开销模型的准确性不足，并且没有考虑显存开销，在实际的应用中会大概率发生内存不足（Out of Memory）的问题，故而仍然具有很大的改进空间。

而与 ZeRO 相关的工作中，不论是 ZeRO、MiCS 还是 ZeRO++ 和 AMSP，在面向大语言模型并行训练的异构性环境下，均存在着潜在的问题和提升的空间。ZeRO 中存在着庞大的通信开销；MiCS 中存在着对模型规模的限制，并且对于异构的网络连接中的高带宽连接利用率低；ZeRO++ 的量化方法引入了潜在的信息损失和模型性能的下降，存在通用性问题；AMSP 针对通信的优化思路同样局限于减少跨节点的通信量，无法充分适应和利用异构集群中存在的不同存储、算力和网络带宽。

2.4 梯度检查点优化

在大规模模型与长序列训练中，为节省显存，常采用梯度检查点（Gradient Checkpointing）技术，即不保存所有层的中间激活，仅在部分层保存，反向传播时对未保存的层进行重新前向计算以得到梯度。全部重计算策略不保存任何中间激活，仅在需要时按需重算，显存占用最低但计算量增加明显。选择重计算策略则选择部分层作为检查点保存激活，在显存与计算之间取得折中。细粒度重计算进一步在算子或更小粒度上决定保存与重算，可结合模型结构和异构设备的显存与算力进行更精细的优化。在异构 GPU 集群上，不同设备显存与算力差异大，将梯度检查点与流水线划分、stage 分配结合，可避免显存溢出并提高整体吞吐，是本文后续优化方向之一。Colossal-Auto[30] 等工作对自动并行与激活检查点进行统一自动化，为面向异构环境的联合优化提供了参考。

第三章 基于锐度比策略的自适应锐度正则化算法 SAMAR

3.1 引言

3.2 SAMAR 算法设计与实现

3.2.1 锐度比策略设计

3.2.2 算法实现

3.3 SAMAR 算法的理论属性

3.3.1 基本假设

3.3.2 收敛性分析

3.4 实验验证

3.4.1 图像分类实验

3.4.2 Hessian Spectrum 分析

3.4.3 损失曲面可视化

3.5 本章小结

第四章 基于退火策略的自适应锐度正则化算法 ARSO

4.1 引言

4.2 ARSO 算法设计与实现

4.2.1 退火策略

4.2.2 算法实现

4.2.3 锐度比策略与退火策略的比较

4.3 ARSO 算法的理论属性

4.3.1 基本假设

4.3.2 收敛性分析

4.4 实验验证

4.4.1 GLUE 基准测试

4.4.2 神经机器翻译实验

4.4.3 损失曲面可视化

4.5 本章小结

第五章 总结与展望

5.1 全文工作总结

5.1.1 主要创新点

5.1.2 研究的局限性

5.2 未来工作展望

5.2.1 理论深化

5.2.2 应用拓展

致 谢

衷心感谢导师 xxx 教授和 xxx 副教授对本人的精心指导。他们的言传身教将使我终生受益。

感谢 NUDTPAPER，它的存在让我的论文写作轻松自在了许多，让我的论文格式规整漂亮了许多。

参考文献

- [1] Miao X, Wang Y, Jiang Y, et al. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism [J]. Proceedings of the VLDB Endowment. 2022, 16 (3): 470–479.
- [2] Vaswani A, Shazeer N, Parmar N, et al. Attention is All You Need [C]. Advances in Neural Information Processing Systems. 2017.
- [3] Kenton J D M W C, Toutanova L K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [J]. 2019, 1: 2.
- [4] Radford A, Wu J, Child R, et al. Language Models are Unsupervised Multitask Learners [J]. OpenAI blog. 2019, 1 (8): 9.
- [5] Raffel C, Shazeer N, Roberts A, et al. Exploring the Limits of Transfer Learning with a Unified Text-to-text Transformer [J]. Journal of Machine Learning Research. 2020, 21 (1): 5485–5551.
- [6] Brown T, Mann B, Ryder N, et al. Language Models are Few-shot Learners [J]. Advances in Neural Information Processing Systems. 2020, 33: 1877–1901.
- [7] OpenAI. GPT-4 Technical Report [J]. arXiv.org. 2023. 2023-03-15.
- [8] Dosovitskiy A, Beyer L, Kolesnikov A, et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale [C]. International Conference on Learning Representations. 2020.
- [9] Zhai X, Kolesnikov A, Houlsby N, et al. Scaling Vision Transformers [J]. 2022: 12104–12113.
- [10] Radford A, Kim J W, Hallacy C, et al. Learning Transferable Visual Models from Natural Language Supervision [C]. International Conference on Machine Learning. 2021: 8748–8763.
- [11] Ramesh A, Pavlov M, Goh G, et al. Zero-shot Text-to-image Generation [C]. International Conference on Machine Learning. 2021: 8821–8831.
- [12] Li J, Li D, Savarese S, et al. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models [J]. International Conference on Machine Learning. 2023: 19730–19742.
- [13] Liu H, Li C, Wu Q, et al. Visual Instruction Tuning [J]. Advances in Neural Information Processing Systems. 2023, 36.
- [14] Alayrac J-B, Donahue J, Luc P, et al. Flamingo: a Visual Language Model for

Few-shot Learning [J]. *Advances in Neural Information Processing Systems*. 2022, 35: 23716–23736.

[15] Zhu D, Chen J, Shen X, et al. MiniGPT-4: Enhancing Vision-language Understanding with Advanced Large Language Models [J]. *arXiv preprint arXiv:2304.10592*. 2023.

[16] Kaplan J, McCandlish S, Henighan T, et al. Scaling Laws for Neural Language Models [J]. *arXiv preprint arXiv:2001.08361*. 2020.

[17] DeepSeek-AI. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism [J]. *arXiv preprint arXiv:2401.02954*. 2024.

[18] Yang A, Yang B, Hui B, et al. Qwen2 Technical Report [J]. *arXiv preprint arXiv:2407.10671*. 2024.

[19] Narayanan D, Shoeybi M, Casper J, et al. Efficient Large-scale Language Model Training on GPU Clusters Using Megatron-LM [C]. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2021: 1–15.

[20] Shoeybi M, Patwary M, Puri R, et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism [J]. *arXiv preprint*. 2020.

[21] Rajbhandari S, Rasley J, Ruwase O, et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models [C]. *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. 2020: 1–16.

[22] Rajbhandari S, Ruwase O, Rasley J, et al. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning [J]. *arXiv preprint*. 2021.

[23] Li S, Zhao Y, Varma R, et al. PyTorch Distributed: Experiences on Accelerating Data Parallel Training [J]. *Proceedings of the VLDB Endowment*. 2020, 13 (12): 3005–3018.

[24] Sergeev A, Del Balso M. Horovod: Fast and Easy Distributed Deep Learning in TensorFlow [J]. *arXiv preprint*. 2018.

[25] Huang Y, Cheng Y, Bapna A, et al. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism [J]. *arXiv preprint*. 2019.

[26] Narayanan D, Harlap A, Phanishayee A, et al. PipeDream: Generalized Pipeline Parallelism for DNN Training [C]. *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 2019: 1–15.

[27] Chen Z, Lepikhin D D, Lee H, et al. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding [J]. 2020.

- [28] Wang G, Qin H, Jacobs S A, et al. ZeRO++: Extremely Efficient Collective Communication for Giant Model Training [J]. arXiv preprint. 2023.
- [29] Micikevicius P, Narang S, Alben J, et al. Mixed Precision Training [C]. International Conference on Learning Representations. 2018.
- [30] Liu Y, Li S, Fang J, et al. Colossal-Auto: Unified Automation of Parallelization and Activation Checkpoint for Large-scale Models [J]. arXiv preprint. 2023.
- [31] Jia X, Jiang L, Wang A, et al. Whale: Efficient Giant Model Training over Heterogeneous GPUs [J]. 2022.
- [32] Park J H, Yun G, Chang M Y, et al. HetPipe: Enabling Large DNN Training on (Whimpy) Heterogeneous GPU Clusters through Integration of Pipelined Model Parallelism and Data Parallelism [C]. 2020 USENIX Annual Technical Conference (USENIX ATC 20). 2020: 307–321.
- [33] NVIDIA Corporation. NVIDIA A100 Tensor Core GPU Architecture [R]. 2020. Ampere Architecture.
- [34] NVIDIA Corporation. NVIDIA H100 Tensor Core GPU Architecture [R]. 2022. Hopper Architecture.
- [35] NVIDIA Corporation. NVIDIA Blackwell Architecture and GPU Roadmap [R]. 2024. GTC 2024.
- [36] Dubey A, Jauhri A, Pandey A, et al. The LLaMA 3 Herd of Models [J]. arXiv preprint arXiv:2407.21783. 2024.
- [37] Rombach R, Blattmann A, Lorenz D, et al. High-resolution Image Synthesis with Latent Diffusion Models [C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022: 10684–10695.
- [38] Ronneberger O, Fischer P, Brox T. U-Net: Convolutional Networks for Biomedical Image Segmentation [C]. Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015. 2015: 234–241.
- [39] Wu S, Fei H, Qu L, et al. Next-GPT: Any-to-any Multimodal LLM [J]. arXiv preprint arXiv:2309.05519. 2023.
- [40] Dong R, Han C, Peng Y, et al. DreamLLM: Synergistic Multimodal Comprehension and Creation [J]. arXiv preprint arXiv:2309.11499. 2023.
- [41] Kingma D P. Adam: A Method for Stochastic Optimization [J]. arXiv preprint arXiv:1412.6980. 2014.
- [42] Kuznetsova A, Rom H, Alldrin N, et al. The Open Images Dataset V4: Unified Image Classification, Object Detection, and Visual Relationship Detection at Scale [J].

International Journal of Computer Vision. 2020, 128 (7): 1956–1981.

[43] Abu-El-Haija S, Kothari N, Lee J, et al. YouTube-8M: A Large-Scale Video Classification Benchmark [J]. 2016.

[44] Korthikanti V A, Casper J, Lym S, et al. Reducing Activation Recomputation in Large Transformer Models [C]. Proceedings of Machine Learning and Systems (MLSys). 2023: 1–20.

[45] Li M. Scaling Distributed Machine Learning with the Parameter Server [C]. Proceedings of the 2014 International Conference on Big Data Science and Computing. Beijing, China, 2014: 1–1.

[46] Kim S, Yu G I, Park H, et al. Parallax: Sparsity-aware Data Parallel Training of Deep Neural Networks [C]. Proceedings of the Fourteenth EuroSys Conference. Dresden, Germany, 2019: 1–15.

[47] Li Z, Zhuang S, Guo S, et al. TeraPipe: Token-Level Pipeline Parallelism for Training Large-Scale Language Models [J]. arXiv preprint. 2021.

[48] Zhang Z, Zheng S, Wang Y, et al. MiCS: Near-linear Scaling for Training Gigantic Model on Public Cloud [J]. Proceedings of the VLDB Endowment. 2022, 16 (1): 37–50.

[49] Chen Q, Hu Q, Ye Z, et al. AMSP: Super-Scaling LLM Training via Advanced Model States Partitioning [J]. arXiv preprint. 2023.

[50] Li D, Wang H, Xing E, et al. AMP: Automatically Finding Model Parallel Strategies with Heterogeneity Awareness [C]. Advances in Neural Information Processing Systems. 2022: 6630–6639.

作者在学期间取得的学术成果

发表的学术论文

- [1] Yang Y, Ren T L, Zhang L T, et al. Miniature microphone with silicon- based ferroelectric thin films. Integrated Ferroelectrics, 2003, 52:229-235. (SCI 收录, 检索号:758FZ.)
- [2] 杨轶, 张宁欣, 任天令, 等. 硅基铁电微声学器件中薄膜残余应力的研究. 中国机械工程, 2005, 16(14):1289-1291. (EI 收录, 检索号:0534931 2907.)
- [3] 杨轶, 张宁欣, 任天令, 等. 集成铁电器件中的关键工艺研究. 仪器仪表学报, 2003, 24(S4):192-193. (EI 源刊.)
- [4] Yang Y, Ren T L, Zhu Y P, et al. PMUTs for handwriting recognition. In press. (已被 Integrated Ferroelectrics 录用. SCI 源刊.)
- [5] Wu X M, Yang Y, Cai J, et al. Measurements of ferroelectric MEMS microphones. Integrated Ferroelectrics, 2005, 69:417-429. (SCI 收录, 检索号:896KM.)
- [6] 贾泽, 杨轶, 陈兢, 等. 用于压电和电容微麦克风的体硅腐蚀相关研究. 压电与声光, 2006, 28(1):117-119. (EI 收录, 检索号:06129773469.)
- [7] 伍晓明, 杨轶, 张宁欣, 等. 基于 MEMS 技术的集成铁电硅微麦克风. 中国集成电路, 2003, 53:59-61.

研究成果

- [1] 任天令, 杨轶, 朱一平, 等. 硅基铁电微声学传感器畴极化区域控制和电极连接的方法: 中国, CN1602118A. (中国专利公开号.)
- [2] Ren T L, Yang Y, Zhu Y P, et al. Piezoelectric micro acoustic sensor based on ferroelectric materials: USA, No.11/215, 102. (美国发明专利申请号.)

公开评阅信息

序号	评阅人	职称	导师类型	工作单位	总分	结论	答辩建议	熟悉程度	备注
1	张三	教授	博导	XXX大学	95.8	达到	无需修改直接答辩	有深入了解	
2	李四	研究员	硕导	XXX大学	95	达到	修改后答辩	有深入了解	
3	王五	教授	博导						
4	赵六	教授	博导						
5	孙六	教授	博导	XXX大学	59	尚未达到	修改后复评	有深入了解	
	孙六	教授	博导	XXX大学	80	尚未达到	无需修改直接答辩	有深入了解	复评结果

说明：

1. 结论选项包括 2 个：“达到博士学位论文要求”、“尚未达到博士学位论文要求”。
2. 答辩建议选项包括 4 个：“无需修改直接答辩”、“修改后答辩”、“修改后复评”、“不予答辩”。
3. 熟悉程度选项包括 3 个：“有深入了解”、“比较熟悉”、“一般了解”。

提醒（正式成文后删除）：

1. 评阅版论文删除此页。
2. 采用双盲评阅方式的学位申请人撰写的学位论文删除此页。
3. 评阅总分无需取整。
4. 工作单位填至学校、科研院所即可。

附录 A 模板提供的希腊字母命令列表

大写希腊字母:

Γ \Gamma	Λ \Lambda	Σ \Sigma	Ψ \Psi
Δ \Delta	Ξ \Xi	Υ \Upsilon	Ω \Omega
Θ \Theta	Π \Pi	Φ \Phi	
Γ \varGamma	Λ \varLambda	Σ \varSigma	Ψ \varPsi
Δ \varDelta	Ξ \varXi	Υ \varUpsilon	Ω \varOmega
Θ \varTheta	Π \varPi	Φ \varPhi	

小写希腊字母:

α \alpha	θ \theta	o o	τ \tau
β \beta	ϑ \vartheta	π \pi	υ \upsilon
γ \gamma	ι \iota	ϖ \varpi	ϕ \phi
δ \delta	κ \kappa	ρ \rho	φ \varphi
ϵ \epsilon	λ \lambda	ϱ \varrho	χ \chi
ε \varepsilon	μ \mu	σ \sigma	ψ \psi
ζ \zeta	ν \nu	ς \varsigma	ω \omega
η \eta	ξ \xi	κ \varkappa	$\digamma$ \digamma
α \upalpha	θ \uptheta	o \mathrm{o}	τ \uptau
β \upbeta	ϑ \upvartheta	π \uppi	υ \upupsilon
γ \upgamma	ι \upiota	ϖ \upvarpi	ϕ \upphi
δ \updelta	κ \upkappa	ρ \uprho	φ \upvarphi
ϵ \upepsilon	λ \uplambda	ϱ \upvarrho	χ \upchi
ε \upvarepsilon	μ \upmu	σ \upsigma	ψ \uppsi
ζ \upzeta	ν \upnu	ς \upvarsigma	ω \upomega
η \upeta	ξ \upxi		

希腊字母属于数学符号类别, 请用\bm命令加粗, 其余向量、矩阵可用\mathbf。